



USC University of
Southern California

ReAct from Scratch

Planning & Reasoning Loops

CSCI 599 — Agentic AI · Lecture 5 / Week 3

Prof. Marco Papa

USC



Where we left off

Lecture 4 closed with a five-technique sweep on one code-review task

- Zero-shot, Few-shot, CoT, configurable thinking, ReAct

The ReAct function in `lecture04_prompting_sweep.py` was the destination

- Today we build it manually, message by message

Why manual: the message shape is the lesson

- `bind_tools` and framework wrappers hide it

Reasoning Before Acting



Chain-of-Thought, what it actually is

A prompt-level technique — no model changes, no fine-tuning

Elicits intermediate reasoning steps before the final answer

- Few-shot examples that show the steps, or zero-shot via instruction

Often helps because the model conditions its answer on the generated intermediate steps

- More tokens between question and answer = more compute applied

Wei et al. 2022 — [arXiv:2201.11903](https://arxiv.org/abs/2201.11903)

Zero-shot CoT

```
messages = [  
  {"role": "system",  
   "content":  
     "Review this code.\n"  
     "Let's think step by step.\n"  
     "For each line, ask:\n"  
     "  what could go wrong?\n"  
     "Then list the issues."},  
  {"role": "user",  
   "content": code},  
]
```

No examples needed

The instruction alone — "Let's think step by step" — can elicit structured reasoning. Kojima et al. 2022 found this useful across arithmetic, symbolic, and commonsense benchmarks.

Same call from Lecture 4 sweep §3.
Kojima et al. 2022 — arXiv:2205.11916

Configurable thinking — same idea, moved into the model

Different model families expose different reasoning controls:

- Gemma 4 — <|think|> control token
- Gemini 2.5 — thinking_budget; Gemini 3.x — thinking_level
- OpenAI reasoning models — reasoning_effort

The API surface determines how reasoning is controlled and returned

CoT vs configurable thinking

CoT – in the prompt

```
msgs = [
    {"role": "system",
     "content":
        "Review this code.\n"
        "Think step by step."},
    {"role": "user",
     "content": code},
]
r = client.chat.completions.create(
    model="gpt-5.5",
    messages=msgs,
)
```

Configurable thinking

```
msgs = [
    {"role": "system",
     "content":
        "Review this code. "
        "Be thorough."},
    {"role": "user",
     "content": code},
]
r = client.chat.completions.create(
    model="gpt-5.5",
    messages=msgs,
    reasoning_effort="xhigh",
)
```

When to reach for configurable thinking

PAPA'S RULE

Configurable thinking moves CoT inside the model. You write less prompt, you pay more compute. Reach for it when authoring CoT chains becomes the bottleneck — not before.

Use CoT prompting when the chain is short and you want to inspect the steps. The visibility matters more than the cost.

Configurable thinking earns its keep on long chains, or when the model is already doing reasoning you're paying for. Stop authoring the chain by hand.

Reasoning and Acting



ReAct — Yao et al. 2022

arXiv:2210.03629 — "ReAct: Synergizing Reasoning and Acting in Language Models"

Interleaves reasoning traces with task-specific actions

- Reason — what should I look at next?
- Act — invoke a tool
- Observe — read the tool result
 - Repeat until the task is done or a bound is hit

The contribution is the interleaving of reasoning and acting

Why a loop changes things

Single-call CoT — one shot, one answer, no feedback

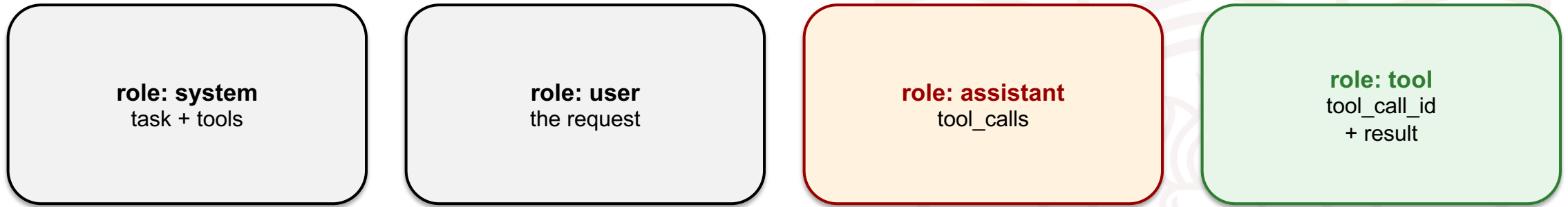
- The model commits to a path based on what's in the prompt

Loop — the model can change its mind after seeing tool output

- Hypothesis → evidence → revised hypothesis → evidence
- The next action can change after observation

Cost: latency grows with model iterations / round-trips, while context grows monotonically

The message shape — abstract view



→ if more tool_calls, loop back

Stop condition: assistant turn with no tool_calls.

The message shape — one turn, concrete

```
# assistant turn — model decides to call a tool
{"role": "assistant",
 "tool_calls": [{
   "id": "call_abc123",
   "function": {
     "name": "git_log",
     "arguments": "{\\"n\\": 3}"
   }]
}

# tool turn — your script provides the result
{"role": "tool",
 "tool_call_id": "call_abc123",
 "content": "a1b2c3 fix login\\nd4e5f6 ..."}

```

The id and tool_call_id must match — that's how the model knows which result goes with which call.

Building the loop — step 1: system prompt + tools

```
TOOLS = [{  
  "type": "function",  
  "function": {  
    "name": "git_log",  
    "description": "Last N commits.",  
    "parameters": {  
      "type": "object",  
      "properties": {  
        "n": {"type": "integer"}},  
      "required": ["n"]}}}]  
  
messages = [  
  {"role": "system",  
   "content": "Review the last 3 "  
    "commits for security issues."},  
  {"role": "user",  
   "content": "Start."}]
```

JSON Schema for each tool.
The model uses these
descriptions to decide
which tool to call when.

Building the loop — step 2: parse tool_calls

```
response = client.chat.completions.create(  
    model="gpt-5.5",  
    messages=messages,  
    tools=TOOLS,  
)  
  
msg = response.choices[0].message  
messages.append(msg.model_dump(  
    exclude_unset=True))  
  
# If no tool_calls, the loop ends here  
if not msg.tool_calls:  
    final_answer = msg.content
```

PEDAGOGY

No bind_tools — we want the message shape visible.

Building the loop — step 3: dispatch + append

```
import json

for call in msg.tool_calls:
    name = call.function.name
    args = json.loads(
        call.function.arguments)
    result = run_tool(name, args)
    messages.append({
        "role": "tool",
        "tool_call_id": call.id,
        "content": result,
    })
```

run_tool is YOUR code.
The model only requests
the call — it never
executes anything.

Building the loop — step 4: stop and bound

```
MAX_ITERS = 7

for _ in range(MAX_ITERS):
    response = client.chat.completions.create(
        model="gpt-5.5",
        messages=messages,
        tools=TOOLS,
    )
    msg = response.choices[0].message
    messages.append(msg.model_dump(
        exclude_unset=True))

    if not msg.tool_calls:
        break    # done

    # else: dispatch tools, continue
```

MAX_ITERS is the safety net.
Without it, a confused model
can call tools forever.

We'll see this in the demo.

The full loop, assembled

```
messages = [
    {"role": "system", "content": SYSTEM_PROMPT},
    {"role": "user", "content": TASK},
]

final_answer = ""
for _ in range(MAX_ITERS):
    response = client.chat.completions.create(
        model=MODEL, messages=messages, tools=TOOLS)
    msg = response.choices[0].message
    messages.append(msg.model_dump(exclude_unset=True))

    if not msg.tool_calls:
        final_answer = msg.content or ""
        break

    for call in msg.tool_calls:
        args = json.loads(call.function.arguments)
        result = run_tool(call.function.name, args)
        messages.append({"role": "tool",
            "tool_call_id": call.id, "content": result})
    else:
        final_answer = "Agent stopped at MAX_ITERS without a "
            "natural final answer."
```

Demo — Commit Review



Demo setup — commit-review agent

Task — review the last 3 commits for security issues

Codebase — sample_code.py from Lecture 4 + fabricated git history

- md5 commit — switched password hash to md5 (the High-severity issue)
- refactor commit — env-driven config loading (benign)
- CLI commit — added --verbose and --password args (a second issue)

Tools — git_log, git_diff, read_file, check_cves

Bound — MAX_ITERS = 7 (tool-call count and path are nondeterministic)

Live — python lecture05_react_manual.py

LIVE DEMO

Switch to terminal.

Watch the trace: reason → act → observe → repeat.

Expected: several tool calls across roughly 4–6 iterations; model behavior isn't deterministic.

Time budget: ~18 minutes including trace discussion.

What we just saw

The reason-act-observe trace, made concrete

- Reason: "I should check what changed recently" → `git_log`
- Observe: three commits → decide to inspect each
- Act: `git_diff` × 3 in one model iteration
- Inspect: `read_file` × 3 in the next model iteration
- Confirm: `check_cves` × 3 in the next model iteration
- The model can request many `tool_calls` in one response; this demo executes them serially

Each iteration is a model round-trip; tool calls are the requests inside that response

Natural stop at iteration 5, well under `MAX_ITERS` = 7

- The bound didn't fire this run — but keep it. Next run may differ.

Multiple tool_calls — one model round-trip, many calls

MESSAGE SHAPE

The model can request several tools in one assistant response. Our manual executor then handles those requests one by one.

The `tool\_calls` field is a list, so one model round-trip can contain multiple tool requests

- `for call in msg.tool\_calls:` reads every requested call
- This demo executes the calls serially; it does not create concurrent workers

Tested run: 10 tool calls across 5 model iterations

- Three requested calls appeared together in iterations 2, 3, and 4
- Concurrent execution is a separate executor design choice

27.66s total wall clock is the measured run —
not evidence of concurrent tool execution.

Failure mode 1 — looping on a non-issue

The model can't always tell when its investigation is complete

- Sees something it doesn't fully understand
- Decides one more tool call will clarify it
- The tool result doesn't clarify it — call again

Didn't fire in today's demo. It will on other runs — behavior isn't deterministic.

Mitigation — MAX_ITEERS, plus a sharper system prompt about stop

- A bound is the hard guarantee; prompting only influences natural stop

Failure mode 2 — context window blows up

Every iteration appends to messages — context grows monotonically

Tool outputs can be large (file contents, log dumps, search results)

Long tool outputs and long-running loops can eventually exhaust the context window

Demo made 10 calls with truncated tool outputs at 4000 chars — that truncation is why it fit

Mitigations:

- Truncate tool outputs at append time (head + tail summary)
- Use a summarizer call to compress old turns when context gets large
- Pick tools whose outputs are bounded by design (line counts, paging)

Failure mode 3 — hallucinated tools

The model emits a `tool_call` for a tool you didn't define

- Often a plausible-sounding name like `list_files` or `search_code`
- Sometimes a misspelling of a real tool — `git_logs` vs `git_log`

Did not fire in our demo — the model stuck to the four registered tools

Guard — explicit unknown-tool check in ``run_tool``

- Return an `ERROR` tool result instead of raising
- Returning the error to the model lets it self-correct in the next turn

Five techniques, four dimensions

Technique	Cost	Latency	Tool access
Zero-shot	Lowest	Fastest	None
Few-shot	Low	Fast	None
CoT (zero-shot)	Low	Slower	None
Configurable thinking	High	Varies by reasoning budget	Model/API dependent
ReAct (loop)	Per-iteration + tools	Grows with model iterations	Yes

ReAct is the only loop in the table — that's what buys you iterative tool use.

When ReAct needs additional planning

PAPA'S RULE

ReAct is the loop. Planning patterns are what you do when the loop alone isn't enough — when the task needs a plan up front, when reasoning paths need to be sampled, when failures need to be remembered across runs.

Next — planning patterns and LangGraph

Self-consistency — sample N reasoning paths, vote on the answer

Tree-of-Thought — branch the reasoning, prune the bad branches

Plan-and-Execute — plan first, then execute, then re-plan

Reflexion — remember what went wrong, retry with that memory

LangGraph — a framework for building these workflows in production

Then tools and protocols — MCP, A2A, ACP/UCP